

# AutoMLP: A Framework for the Acceleration of Multi-Layer Perceptron Models on FPGAs for Real-Time Atrial Fibrillation Disease Detection

Chao Chen , Bruno da Silva , *Member, IEEE*, Chenxi Yang , *Member, IEEE*, Caiyun Ma , *Graduate Student Member, IEEE*, Jianqing Li , and Chengyu Liu , *Senior Member, IEEE*

**Abstract**—Cardiovascular diseases are a leading cause of death globally, and atrial fibrillation (AF) is a common arrhythmia that affects many people. Detecting AF in real-time using hardware acceleration can prompt timely medical intervention. Multi-layer perceptron (MLP) has demonstrated the ability to detect AF accurately. However, implementing MLP on Field-Programmable Gate Array (FPGA) for real-time detection poses challenges due to the complex hardware design requirements. This study presents a novel framework for generating hardware accelerators to detect AF in real-time using MLP on FPGA. The framework automates evaluating MLP model topology, data type, and bit-widths to generate parallel acceleration. The generated solutions are evaluated using two AF datasets, PhysioNet MIT-BIH atrial fibrillation (AFDB) and China Physiological Signal Challenge 2018 (CPSC2018), regarding execution time, resource utilization, and accuracy. The evaluation results demonstrate that the hardware MLP can achieve a speedup higher than  $1500\times$  and around  $25000\times$  lower energy consumption than an embedded CPU. These satisfactory results prove the framework's suitability and convenience for the online detection of AF in an accelerated and automatic way through FPGA hardware implementation.

**Index Terms**—Multi-layer perceptron network (MLP), machine learning, field-programmable gate array (FPGA), atrial fibrillation (AF), high-levels synthesis (HLS), framework.

Manuscript received 14 March 2023; revised 12 May 2023 and 4 July 2023; accepted 14 July 2023. Date of publication 26 July 2023; date of current version 9 January 2024. This work was supported in part by the National Natural Science Foundation of China, under Grants 62171123, 62071241, 62201144, and 62211530112, in part by the China Scholarship Council under Grant 202106090175, in part by the Natural Science Foundation of Jiangsu Province under Grants BK20210208 and BE2022160, in part by the Young Elite Scientists Sponsorship Program by CAST under Grant 2022QNRC001, and in part by the Fundamental Research Funds for the Central Universities under Grant 2242022R40026. This paper was recommended by Associate Editor P. Motto Ros. (Corresponding authors: Bruno da Silva; Jianqing Li; Chengyu Liu.)

Chao Chen is with the School of Instrument Science and Engineering, Southeast University, Nanjing 210096, China, and also with the Department of Electronics and Informatics (ETRO), Vrije Universiteit Brussel (VUB), 1050 Brussels, Belgium (e-mail: 230199170@seu.edu.cn).

Bruno da Silva is with the Department of Electronics and Informatics (ETRO), Vrije Universiteit Brussel (VUB), 1050 Brussels, Belgium (e-mail: bdasilva@etrovub.be).

Chenxi Yang, Caiyun Ma, Jianqing Li, and Chengyu Liu are with the School of Instrument Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: chenxiyang@seu.edu.cn; 230208850@seu.edu.cn; lj@seu.edu.cn; chengyu@seu.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TBCAS.2023.3299084>.

Digital Object Identifier 10.1109/TBCAS.2023.3299084

## I. INTRODUCTION

ATRIAL fibrillation (AF) is a cardiovascular disease characterized by a noticeably irregular heart rate [1]. AF is common in the elderly and increases the risk of stroke and mortality [2]. Currently, AF affects approximately 2.2 million Americans and 4.5 million people in the European Union [3], [4], [5]. Meanwhile, the high proportion of asymptotic AF leaves a high burden on healthcare workers [6]. The Holter recording monitor, a 72-hour ambulatory electrocardiography (ECG) monitoring widely used for AF diagnosis [7], [8], represents a heavy workload for doctors and nurses. Automating AF detection would save medical and nursing staff time, providing timely medical intervention to other potential patients and high-risk groups.

Machine learning (ML) techniques have been adopted as support for medical diagnosis, including AF detection. In fact, ML has advantages in AF detection when trained with sufficient data [9], [10], [11], [12], achieving similar or even better performance than human experts [13], [14] while saving the valuable time of health care providers working on AF diagnosis [15], [16], [17]. Popular ML algorithms such as Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) [15], Multi-Layer Perceptron Networks (MLP) [18], [19], [20], and Support Vector Machine (SVM) [18], have been proposed in the past. However, training these ML techniques demands a large amount of biomedical data to achieve acceptable accuracy. With the development of the Internet of Things (IoT), wearable devices such as smartwatches, bands, and stickers have produced a large amount of biomedical data to help AF detection beyond the Holter [21]. Although such devices promise continuous monitoring, the time demand of the algorithms for autonomous AF recognition has become a concern for healthcare providers and researchers [9], [10], [22]. Unfortunately, wearable devices, particularly those with low-power Central Processing Units (CPUs), cannot support a Deep Learning (DL) model well. The MLP models, on the other hand, are low compute-demanding ML algorithms that work well with both small and large datasets. Furthermore, compared to a Deep CNN model, the MLP model can deliver similar or even better accuracy [18].

Besides their low computational demand, MLP models require hardware acceleration to achieve a real-time response. When considering embedded devices, Field-Programmable

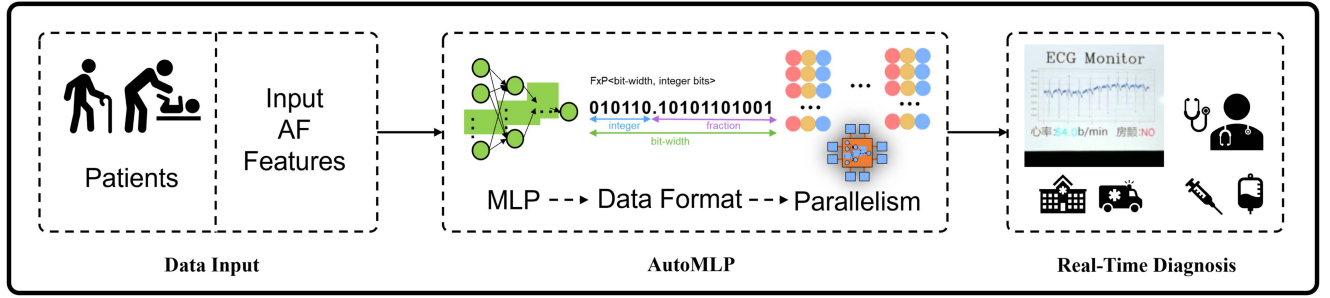


Fig. 1. A real-time embedded AF monitor system is possible thanks to the use of AutoMLP to generate and deploy accelerated MLP models on an FPGA.

Gate Arrays (FPGAs) provide low power consumption and high performance since they can implement performance-efficient architectures. Because of the parallel computation structure, these configurable hardware platforms accelerate ML algorithms, leading to more power-efficient solutions than the ones implemented on CPUs or graphics processing units (GPUs) [23], [24], [25], [26]. This reconfigurable technology has gained interest in ML acceleration because it provides high-computation for parallel algorithms [27], [28], [29] while achieving high-power efficiency [30], [31].

Model deployment is the final step in the machine learning workflow. Given the diversity of end devices, automated frameworks such as AutoML [32], Auto-Keras [33], and Auto-sklearn [34] have been developed to streamline model deployment. These tools are primarily aimed at software implementation [35]. However, the challenge of hardware design remains, and there is currently a lack of automated frameworks for deploying models on hardware, especially FPGAs [36], [37], [38], [39], [40], [41].

MLP implemented on FPGAs reduces time latency and power consumption compared to other technologies [42]. Although many researchers use FPGA to implement MLP models with parallel optimizations [43], [44], they are often simply neural network detector implementations. One main reason is the hard design effort to fully exploit the FPGA's unique characteristics. Such design effort limits the MLP complexity during the design-space exploration.

This work presents a general analysis framework for MLP-based AF detectors on FPGAs. The MLP implementation over FPGA has more design considerations than the simple porting of the model structure. The bit-widths for the data representation, parallelism strategies, architectures, and impact on latency and resources must all be carefully considered. This work investigates model topology, bit-widths of data, and parallelism architectures step by step to explore the design space for MLP detectors on FPGAs. The presented development toolchain leads to quick implementations of MLP-based AF detectors on FPGAs.

The main contributions of this work are as follows:

- A novel framework for MLP implementation on FPGAs towards real-time AF detection. The proposed framework integrates model topology analysis, data bit-widths simulation, and comprehensive comparison for parallel architectures.

- An evaluation of the factors that influence the performance of MLPs implemented on FPGAs, including resource usage, latency, and accuracy, and identification of optimal design approaches to improve efficiency and accuracy.
- The acceleration of existing MLP models and novel proposed MLP models for automatically detecting AF.

The principles of MLP and its adoption for AF detection are described in Section II. The proposed framework is described in Section III. The evaluation results of implementation on CPU and FPGA are performed in Section IV, and the discussion of the results is done in Section V. Finally, the conclusions are drawn in Section VI.

## II. BACKGROUND

This section provides an overview of the background of AF and the use of MLP on FPGAs (Fig. 1).

### A. AF Arrhythmia

AF is a cardiac disorder characterized by an irregular or rapid heartbeat, which can cause symptoms such as heart palpitations, fatigue, dizziness, shortness of breath, and chest pain. AF arises when the electrical signals that coordinate the contraction of the heart's atria become disorganized, leading to an irregular and rapid heart rate. Risk factors for AF include high blood pressure, sleep apnea, and a family history of AF. Early detection of AF is crucial for preventing serious complications, such as stroke and heart failure, and maintaining overall cardiovascular health. Therefore, initiating treatment promptly to mitigate these potential complications is essential.

### B. Features

The most critical and significant component of ECG signals is R peak [53], [54]. P-wave and f-wave are hard to detect, especially for noisy wearable ECG devices. The absolute RR interval irregularity measurement is the most accepted AF detection way [55]. Previous research [16], [54] extracts features from 10 s RR interval to measure the heart rate complexity. Mixed information entropy and statistical domain features are used. Sample entropy (SampEn), Coefficient of SampEn (CosEn), and Entropy for AF (AFEn) are information entropy features. SampEn examines irregularities in data series [56], which is stable for complexity analysis and avoids bias in information entropy

TABLE I  
RELATED WORKS ON AF DETECTION METHODS AND THEIR REAL-TIME CHARACTER

| Reference | Year | Datasets              |          | Platforms                       | Models  | Accuracy | Time    |
|-----------|------|-----------------------|----------|---------------------------------|---------|----------|---------|
|           |      | Train                 | Test     |                                 |         |          |         |
| [45]      | 2018 | AFDB, etc.            | -        | Intel TM i7-8550U CPU           | SampEn  | 67.56%   | 0.15 ms |
| [45]      | 2018 | AFDB, etc.            | -        | Intel TM i7-8550U CPU           | FuzzyEn | 73.04%   | 0.49 ms |
| [45]      | 2018 | AFDB, etc.            | -        | Intel TM i7-8550U CPU           | CosEn   | 89.73%   | 1.58 ms |
| [45]      | 2018 | AFDB, etc.            | -        | Intel TM i7-8550U CPU           | AFEn    | 94.80%   | 1.47 ms |
| [46]      | 2019 | MITDB [47]            | -        | ARM Cortex A7 CPU               | LSTM    | -        | 31.2 ms |
| [48]      | 2020 | AFDB                  | -        | Intel Core i9 CPU, 11 GB GPU    | CS-CNN  | 96.25%   | 26 ms   |
| [49]      | 2020 | MITDB                 | -        | FPGA                            | MLP     | 95%      | -       |
| [50]      | 2020 | Pima Indians Diabetes | -        | -                               | MLP     | 97.76%   | 1.12 s  |
| [51]      | 2021 | AFDB                  | AFDB     | Intel i7 CPU, GeForce 920MX GPU | MLP     | 80.67%   | -       |
| [52]      | 2022 | CPSC2017              | CPSC2017 | -                               | SVM     | 97.25%   | 2.46 s  |

computation. The short series' threshold choice is solved by the proposed CosEn [57]. AFEn, a specially developed algorithm for AF detection, introduces a fuzzy function in similarity pairs [45]. Statistical features of RR interval could provide independent information in AF detection, except for entropy features [45], [57]. The statistical domain features include mean RR intervals of the episode (mRR), maximum RR intervals (maxRR), minimum RR intervals (minRR), and median heart rate (medHR).

### C. Classifiers

The entropy and statistical features for the RR interval are significant and have also been validated in previous research. The mentioned research compares more than 50 features [58], [59], [60], [61], and the features used in this study are significant because of the complexity and irregularity of RR intervals in AF. Another benefit of deploying the MLP model is the stable discriminative ability for well-chosen features [59], [60].

As shown in Table I, while individual features are simple, they may only sometimes suit the dataset. In contrast, DL methods require massive training data and long execution times and rely on GPU resources. By accepting hybrid features, MLP models can achieve higher accuracy than previous individual feature approaches. Furthermore, MLP models are less complex and suitable for embedded CPUs.

### D. Multi-Layer Perceptron Network

An MLP is a computationally intensive process with layers of interacting artificial neural cells. These cells interact with cells in neighboring layers but are not connected to cells in the same layer. A hardware process engine (PE) can accelerate independent computations. Computation is transferred from an input feature vector to generate a diagnostic result through this network with intensive matrix computation in the AF detection inference.

The forward inference of an MLP can be represented by the following (1):

$$net_i^k = \sum_{j=0}^n w_{i,j}^k * y_j^{(k-1)} + b_i^k \quad (1)$$

for each layer with a pre-trained model parameter matrix  $W$ , the computation for the  $i$ th cell in the current layer and the  $j$ th cell in the previous layer (or the  $j$ th element in the input vector) is

### Algorithm 1: MLP: The Forward Process.

---

```

1: input vector :  $X = [x_0 \ x_1 \ \cdots \ x_{n-1}]$ 
2: weights :  $W_{m,n} = \begin{bmatrix} w_{0,0} & \cdots & w_{0,n-1} \\ \vdots & \vdots & \vdots \\ w_{m-1,0} & \cdots & w_{m-1,n-1} \end{bmatrix}$ 
3: bias :  $B = [b_0 \ b_1 \ \cdots \ b_{m-1}]$ 
4:  $L1$ : //Outer loop
5: for  $i = 0; i < m; i++$  do
6:    $y_i = 0$ ;
7:    $L2$ : //Inner loop
8:   for  $j = 0; j < n; j++$  do
9:      $y_i = y_i + w_{i,j} * x_j$ ;
10:  end for
11:   $y_i = y_i + b_i$ ;
12: end for

```

---

performed by adding a bias term to the result. The details of this algorithm are outlined in Algorithm 1, which demonstrates the matrix computation of the forward process using a nested loop structure.

Multiple PEs of MLP, consisting of a forward matrix operation and a ReLU activation function, are used to classify AF events, as depicted in Fig. 2. The MLP comprises multiple artificial neural cells per layer, with each cell in a layer connected to others in the adjacent layer. The neural network computation occurs between layers, as illustrated in Fig. 3, which shows the traversal of parameters during computation between two layers. Matrix operations are the most computationally intensive aspect of this process. In the design of MLP on FPGAs, the main goal is to maximize the reuse of multipliers and adders or to increase the number of these resources to improve efficiency.

## III. METHODS

The proposed framework involves model topology analysis, bit-widths simulation and profiling, and parallelism design as depicted in Fig. 4. 1) The *compiler* analyzes model topology and converts it into C/C++ kernels, which could run on the underlying CPU and be transformed into FPGA design. 2) The *simulator & profiler* estimate model performance with the data type of different bit-widths. 3) In design-space exploration on FPGA, the proposed framework compares the run time of clock

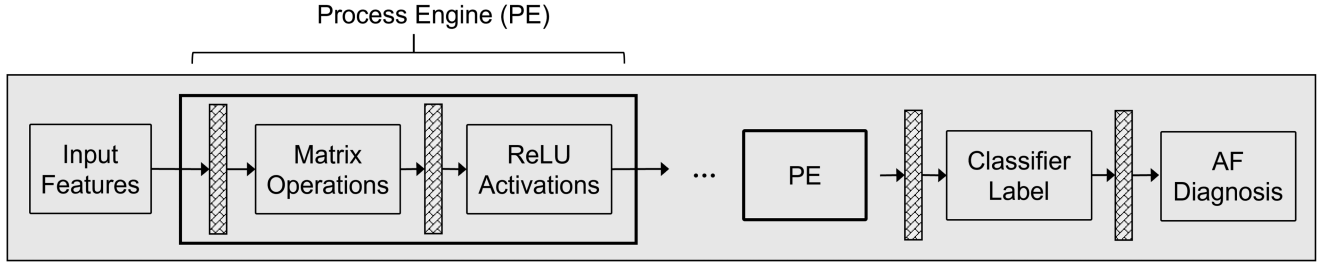


Fig. 2. Data flow of the MLP implementation on the FPGA.

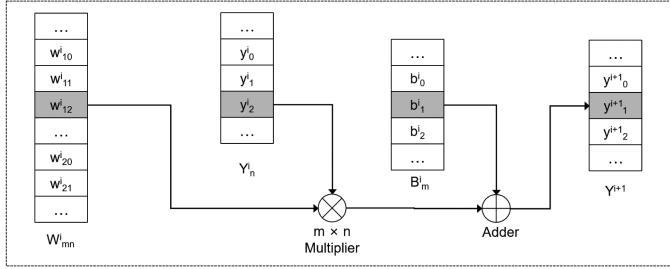


Fig. 3. The detail of matrix computation in a neural network on FPGA.

cycles, resource utilization on storage space, compute units, and logic instances with parallel architectures implemented on FPGA.

#### A. Model Topology - Compiler to Underlying Design

MLP models implemented on FPGAs require physical resources, and their computation time depends on the model's topological structure, including the number of hidden layers, nodes, and activation functions. The topology of an MLP model can affect its storage size and the number of data processing engines needed on the FPGA. The number of hidden layers and nodes is a crucial hyperparameter for MLP models and is influenced by the training results and application conditions.

A Python script parses the model saved in “.pt” format, extracting the structure, weights, and activation parameters. These are then written into arrays in a C/C++ file. The inference can be reconstructed from the topology dimensions and type specified in the C/C++ file. The number of hidden layers and nodes determines the number of operations performed on FPGA. The dimension of the matrix determines the counter value controlling loops. The results are shown in Section IV-B1.

#### B. Adjusted Bit-widths - Simulation & Profiling

Floating point (FP) data is one of the most used data types in general computers because it can represent infinitely small or large numbers because the decimal or fractional point can shift left or right depending on the data. Fixed point (FxP) data, on the other hand, has a fixed fractional point that does not move, making it simple to design on FPGA in many situations. FxP data divides information into integer and fractional parts. For instance, FxP <8,4> represents eight bits with four integer bits (one sign bit included).

The significance of bits can differ significantly depending on several factors. These factors include the number of significant digits, the position of the decimal point, and the sign of the number. A limited number of bits may be sufficient to represent the original model. Testing an MLP model on an FPGA can be time-consuming and require high development efforts. Therefore, it is recommended to use advanced software simulation to evaluate the model's performance before implementing it on an FPGA.

The MLP-based real-time detector accelerated on an FPGA has a configurable bit-width, unlike general processors such as CPUs or GPUs, which typically have fixed 8, 16, 32, or 64 bit-widths to meet the requirements of general computation tasks. However, not all bits are necessary or equally significant in a model. Resource utilization can be saved, and time latency improved by reducing unnecessary bits on the FPGA. The framework's script file traverses configuration for bit-width, invokes C simulation in the FPGA development platform via the command line, and uses input datasets for accuracy profiling. This study evaluates the necessary bits for integer and fractional parts separately. The proposed framework conducts experiments to evaluate the performance of estimated MLP models with chosen bit-widths and presents the results in Section IV-B2.

#### C. Parallel Optimization - Design-Space Exploration

After model compilation, bit-widths simulation, and profiling, it is vital to consider parallel design techniques on FPGA for the MLP module.

As shown in Algorithm 1, the matrix in an MLP has two dimensions, and the matrix computation is a dual-loop process. The inner loop, named L2, updates the node's value and is repeated by the outer loop, L1, for all neural nodes within the same layer. The Pipeline or Unroll strategies can optimize the computation-intensive inner loop operation on FPGA. The Pipeline strategy processes sequential data in the same circuit simultaneously, increasing throughput without using extra circuits. The Unroll strategy uses multiple copies of the same circuit to process data simultaneously, improving processing speed. These two optimization strategies can also be combined for the dual-loop process. For example, the inner loop can be unrolled while the outer loop is pipelined ( $L_{1P} L_{2U}$ ), or both the inner and outer loops can be unrolled to maximize the speedup ( $L_{1U} L_{2U}$ ). Their timing illustration is depicted in the following figures: Figs. 5–7.



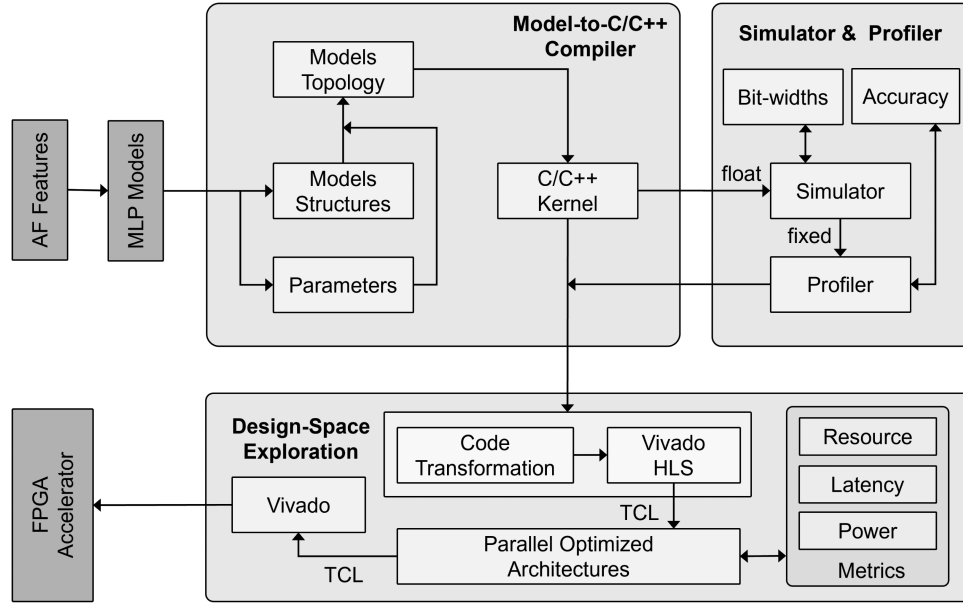


Fig. 4. Overview of the deployment implementation flow. The pre-trained MLP model is compiled into a C/C++ kernel for bit-width simulation and performance profiling. The C/C++ code is integrated for FPGA design using HLS and Vivado tools. The components are abstracted for common users.

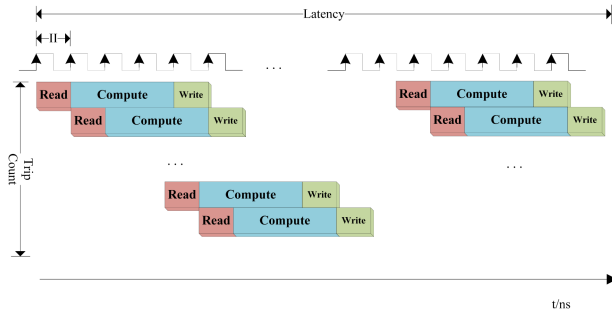


Fig. 5. The illustration of pipeline timing.

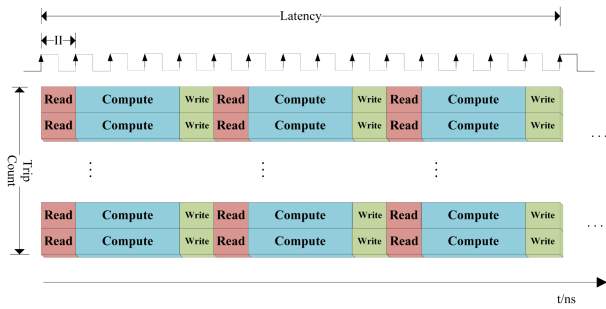


Fig. 6. The illustration of unroll timing.

$L_{2P}$ : As shown in Algorithm 1, the inner loop L2 is the most frequently called process in MLP computation. Using the knowledge that a large logic block can be divided into smaller sub-circuits using registers in pipeline strategy, the sub-circuits in pipeline optimization can work independently. As shown in Fig. 5, each sub-circuit can accept new input data while the next sub-circuit simultaneously processes the data from the previous iteration. The timing for the pipeline optimization is illustrated in Fig. 5.

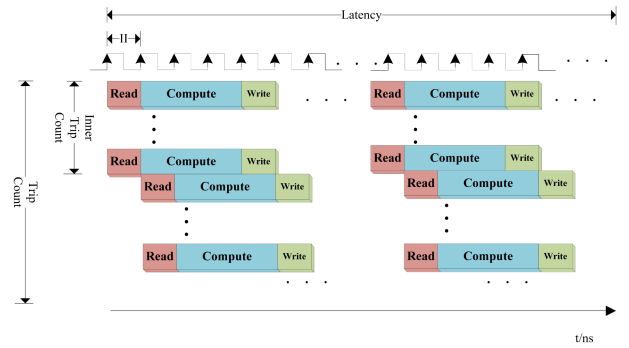


Fig. 7. The illustration timing of combined pipeline and unroll optimization architecture.

$L_{2U}$ : Multiple copies of the same process engine can unroll the operations in the inner loop L2. There is no interval delay between these operations, and all process engines read, compute, and write data in the same clock cycle. Fig. 6 illustrates the timing for the unroll optimization.

$L_{1P}L_{2U}$ : In this architecture, unroll and pipeline techniques are combined to optimize the performance of the outer loop L1. Specifically, the inner loop L2 was unrolled, allowing the processor to overlap the execution of the read, compute, and write operations within the loop. Additionally, the outer loop L1 was optimized using a pipeline, enabling the simultaneous processing of data in sub-circuits within the inner loop. As depicted in Fig. 7, this allowed the processor to read, compute, and write data concurrently, potentially improving the performance of the loop.

$L_{1U}L_{2U}$ : This architecture implements a loop-based architecture for updating the information of neural nodes in a neural network. The inner loop L2 is responsible for updating the information of a single node, while the outer loop L1 updates all

TABLE II  
LABELS AND FEATURES VALUE DISTRIBUTION OF THE DATASETS

| Dataset  | Label distribution |       | Features (mean $\pm$ std) |                 |                 |                 |                 |                 |                 |
|----------|--------------------|-------|---------------------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
|          | Non-AF             | AF    | CosEn                     | SampEn          | AFEn            | mRR             | minRR           | maxRR           | medHR           |
| AFDB     | 33450              | 33450 | -1.34 $\pm$ 1.00          | 0.80 $\pm$ 0.56 | 3.76 $\pm$ 0.33 | 0.75 $\pm$ 0.18 | 0.60 $\pm$ 0.22 | 0.93 $\pm$ 0.24 | 1.43 $\pm$ 0.37 |
| CPSC2018 | 902                | 902   | -1.29 $\pm$ 1.12          | 1.13 $\pm$ 0.47 | 3.15 $\pm$ 0.28 | 1.40 $\pm$ 0.33 | 1.15 $\pm$ 0.43 | 1.75 $\pm$ 0.34 | 0.78 $\pm$ 0.24 |

nodes in the layer. This architecture allows for the simultaneous update of all nodes in the same layer, improving the network's performance. To further increase the parallelism of the system, dual unrolling of the inner loop L2 is used, which allows for the concurrent execution of multiple iterations of the loop. This technique can improve the network's performance by unrolling the overhead of loop control instructions.

The generated hardware files, which consist of modified C/C++ files with a data transfer interface, are used as input in the Vivado tools. The architectures are designed by combining loop operations, and parallel commands are assigned to these loops. The framework executes a script in the command line development environment to automatically generate an accelerator. Once done, the IP can be imported into Vivado for application design.

#### IV. RESULTS

Our framework is evaluated by implementing existing MLP models and novel ones, which are proposed for the detection of AF. In addition, the capabilities of our framework are tested for evaluating an MLP topology analysis, data representation comparison, and acceleration architectures. Performance metrics used in these analyses include FPGA resource utilization, computation latency, and accuracy. Through our experiments, we aim to identify the factors that influence the performance of the MLP model implemented on FPGAs and to evaluate the optimal design approaches for improving their efficiency and accuracy.

##### A. Experiment Setup, Datasets and Models

1) *Platforms and Tools*: In this experiment, a general AI training environment, Pytorch, is used in the Python IDE Jupyter Notebook to train MLP models. This study utilizes AF features extracted from two widely used datasets [62], [63], [64] for MLP model training. The FPGA development tools used for this research are Xilinx HLS and Vivado 2019.1 (except for bit-widths simulation, which uses HLS 2021.2). Tool Command Language (TCL) automates the design process on FPGA. The Xilinx PYNQ-Z1 board (Fig. 9), a Zynq 7020 platform, featured an FPGA and an ARM CPU. The FPGA part with 280 Block Random Access Memory (BRAM), 220 Digital Signal Processing Elements (DSP), 106,400 Flip Flop (FF), and 53,200 Look Up Table (LUT) was utilized in this study at a frequency of 100 MHz. The experiments were conducted on the low-end CPU (ARM-Cortex 9), high-performance CPU (Intel Xeon E5), and the FPGA standalone. The FPGA implementation without parallel design is used as a baseline (BL) to measure time delay and resource utilization.

TABLE III  
DESCRIPTION AND ACCURACY OF THE MODELS IMPLEMENTED WITHIN THE PROPOSED FRAMEWORK

| Reference | Models      | Topology       | Accuracy(FP) | Accuracy(FxP<8,4>) |
|-----------|-------------|----------------|--------------|--------------------|
| [65]      | MLP_Saric   | (7-46-2)       | 98.94%       | 94.73%             |
| [51]      | MLP_Borghi  | (7-60-60-60-2) | 97.39%       | 53.66%             |
| Proposed  | MLP_3Layers | (7-16-2)       | 98.00%       | 95.29%             |
| Proposed  | MLP_5Layers | (7-12-16-12-2) | 98.23%       | 96.78%             |

2) *Datasets*: The two datasets used for performance profiling of the generated MLPs are detailed in Table II. Because of the irregularity of AF rhythm, the entropy features and RR interval statistical features are extracted [16], [54].

- MIT-BIH atrial fibrillation (AFDB) [62] is a widely used AF dataset with 25 long-term (10 hours) ECG records with paroxysmal AF. The data is sampled with 250 points per second. AFDB has ECG records of two channels and labels of AF rhythms.
- The China Physiological Signal Challenge 2018 (CPSC2018) [64] is a comprehensive database from 11 hospitals containing nine ECG types with normal and AF-labeled data. In CPSC 2018 dataset, ECG signals are sampled at 500 Hz. The first of the twelve leads ECG was chosen from normal and AF records.

3) *Metrics*: The proposed framework addresses the pressing concerns of resource consumption, computation latency, and model accuracy (Fig. 8).

- FPGA resources include configurable components such as BRAM, DSP, FF, and LUT. BRAM serves as a memory resource for data buffering and storage, DSP performs mathematical operations, FF acts as an intermediate value register, and LUT is a logic resource for combinational logic functions.
- Latency, referring to the time taken for FPGA or CPU to complete a task or operation, is also a critical consideration.
- In this deployment framework, accuracy serves as the key metric for quantization. True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) are counted for AF detection, and accuracy is calculated as (TP+TN)/(TP+TN+FP+FN). The proposed framework aims to minimize the accuracy loss of an MLP model during deployment.
- Power consumption is the quantity of electrical power a device consumes over time, commonly measured in watts (W). Based on the model's latency and power consumption, the total energy utilized by one AF categorization, measured in joules (J), can be obtained. These metrics are critical in determining electronic devices' and systems' efficiency and sustainability.

4) *Models*: Table III shows the details of the MLP models under evaluation. Two of the MLP models are taken from the

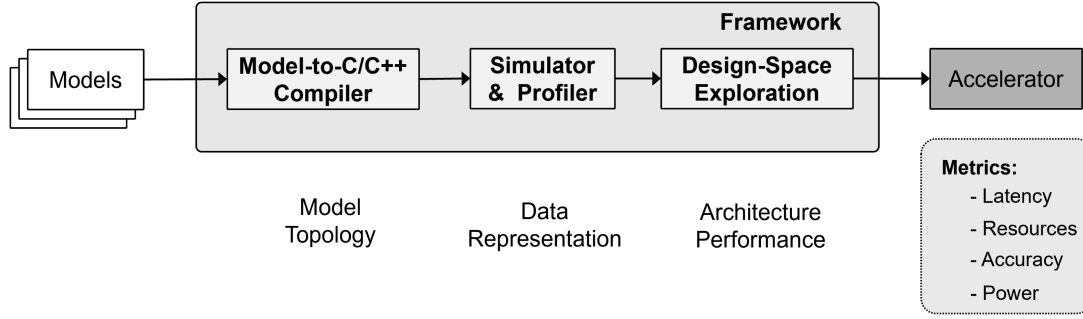


Fig. 8. Illustration of experiments performed to evaluate the proposed framework.

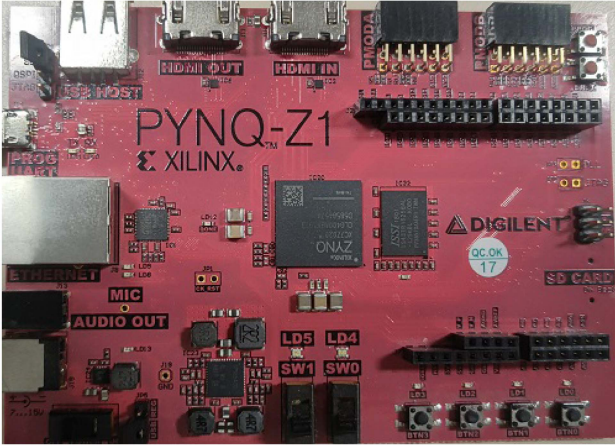


Fig. 9. Xilinx Pynq-Z1 Board is used for Real-time AF detection.

literature. They are used to demonstrate how our framework can implement existing MLP models and to be used as a reference for comparison. Two other MLP models are proposed and evaluated to accelerate AF detection on FPGAs.

The PyTorch training framework is used to pre-train the models. The AFDB dataset is segmented into training and validation sets, and the CPSC2018 dataset is used for testing. The model selection criterion for the proposed framework is based on reducing the training loss and no further reduction of the validation loss. The optimizer is Adam, and the loss function is Cross Entropy.

Since the proposed framework evaluated the data representation format, our evaluation targets different bit-widths for the data representation. Although the FP format data is 32 bits by default, the FxP format data can present a variable bit-width. FxP is represented as FxP  $\langle x+y, x \rangle$ , where  $x$  is the bit-widths of the integer part, and  $y$  is the bit-widths of the fractional part. The accuracy is measured in both data formats.

- *MLP\_Saric*: A single hidden layer MLP model is proposed in [65] for epileptic seizures. This model comprises 46 hidden nodes with 98.94% accuracy on CPSC2018.
- *MLP\_Borghi*: The proposed MLP model in [51] comprises 180 nodes and three hidden layers to discover AF. This study rebuilds the model using the hidden layer's identical 60-60-60 structure with 97.39% accuracy on CPSC2018.

- *MLP\_3Layers*: Three layers of MLP with one single hidden layer: seven nodes in the input features layer, 16 in a hidden layer, and two in the output layer. The accuracy of MLP\_3Layers on AFDB is 93.29% and 98.00% on CPSC2018.
- *MLP\_5Layers*: An MLP model comprises five layers with 12,16,12 nodes in the hidden layer. The accuracy of MLP\_5Layers on AFDB is 94.20% and 98.23% on CPSC2018.

## B. Framework Evaluation

The main components of our framework are exploited to perform different analyses regarding the MLP model topology, the impact of the data representation, and the achievable acceleration. Fig. 8 depicts the relation between our experiments and the internal components of our framework.

1) *Evaluation of the Model Topology*: Our framework facilitates the evaluation of different MLP topologies. This first analysis consists in varying the number of hidden layers and nodes separately. For evaluating the hidden layers, 12 hidden nodes are considered, while only one hidden layer is involved in the variation of the number of hidden nodes. Metrics such as latency and resource consumption are used to evaluate different topologies with the impact of data representation (FP or FxP data format). The architectures are not optimized for this model topology analysis. Therefore, the operations are executed in sequential order. In addition, the FxP data format selected for the model topology evaluation has a bit-width of 8 bits, using 4 bits for the integer and fractional parts. The selection of this particular bit-width is explained in Section IV-B2.

Tables IV and V show the impact on latency and resource usage based on the model's data representations when increasing the number of hidden layers/nodes of the MLP model. FxP data formats, such as the FxP  $\langle 8.4 \rangle$  chosen for our evaluation, require fewer bits to represent the weight parameters of the MLP than the FP data format, as it requires 32 bits in single precision. The data format directly affects latency and resources, especially memory.

Latency increases with the number of hidden layers/nodes because matrix inference operations increase. The FxP data format is faster because one iteration in the matrix calculation consumes only three clock cycles, whereas using the FP data format consumes about 11 clock cycles. Since data transfer and

TABLE IV  
TIME LATENCY AND RESOURCE CONSUMPTION ON AN FPGA WITH A VARYING NUMBER OF HIDDEN LAYERS

| # Hidden Layers | FP                     | FxP<8,4> | Ratio | FP     |       |           |            | FxP<8,4> |         |           |             |
|-----------------|------------------------|----------|-------|--------|-------|-----------|------------|----------|---------|-----------|-------------|
|                 | Latency (clock cycles) |          |       | BRAM   | DSP   | FF        | LUT        | BRAM     | DSP     | FF        | LUT         |
| 1               | 1,335                  | 382      | 3.49× | 3(1%)  | 8(4%) | 2,506(2%) | 3,951(7%)  | 1(0%)    | 22(10%) | 5,716(5%) | 11,787(22%) |
| 2               | 3,040                  | 851      | 3.57× | 4(1%)  | 8(4%) | 2,749(3%) | 4,261(8%)  | 1(0%)    | 23(10%) | 5,788(5%) | 11,988(23%) |
| 3               | 4,745                  | 1,320    | 3.59× | 5(2%)  | 8(4%) | 2,992(3%) | 4,555(9%)  | 1(0%)    | 24(11%) | 5,860(6%) | 12,189(23%) |
| 4               | 6,450                  | 1,789    | 3.61× | 6(2%)  | 8(4%) | 3,235(3%) | 4,863(9%)  | 1(0%)    | 25(11%) | 5,934(6%) | 12,390(23%) |
| 5               | 8,155                  | 2,258    | 3.61× | 7(3%)  | 8(4%) | 3,478(3%) | 5,163(10%) | 1(0%)    | 26(12%) | 6,006(6%) | 12,588(24%) |
| 6               | 9,860                  | 2,727    | 3.62× | 8(3%)  | 8(4%) | 3,721(3%) | 5,448(10%) | 1(0%)    | 27(12%) | 6,075(6%) | 12,787(24%) |
| 7               | 11,565                 | 3,196    | 3.62× | 9(3%)  | 8(4%) | 3,964(4%) | 5,733(11%) | 1(0%)    | 28(13%) | 6,148(6%) | 12,987(24%) |
| 8               | 13,270                 | 3,665    | 3.62× | 10(4%) | 8(4%) | 4,207(4%) | 6,018(11%) | 1(0%)    | 29(13%) | 6,219(6%) | 13,190(25%) |
| 9               | 14,975                 | 4,134    | 3.62× | 11(4%) | 8(4%) | 4,450(4%) | 6,303(12%) | 1(0%)    | 30(14%) | 6,290(6%) | 13,404(25%) |
| 10              | 16,680                 | 4,603    | 3.62× | 12(4%) | 8(4%) | 4,693(4%) | 6,588(12%) | 1(0%)    | 31(14%) | 6,363(6%) | 13,604(26%) |

The ratio is the speedup of FxP < 8,4 > and FP.

TABLE V  
TIME LATENCY AND RESOURCE CONSUMPTION ON AN FPGA WITH A VARYING NUMBER OF HIDDEN NODES IN ONE-HIDDEN-LAYER MLP

| # Hidden Nodes | FP                     | FxP<8,4> | Ratio | FP     |       |           |           | FxP<8,4> |         |           |             |
|----------------|------------------------|----------|-------|--------|-------|-----------|-----------|----------|---------|-----------|-------------|
|                | Latency (clock cycles) |          |       | BRAM   | DSP   | FF        | LUT       | BRAM     | DSP     | FF        | LUT         |
| 4              | 459                    | 146      | 3.14× | 2(1%)  | 8(4%) | 2,557(2%) | 3,951(7%) | 1(0%)    | 22(10%) | 5,811(5%) | 12,019(23%) |
| 8              | 899                    | 262      | 3.43× | 3(1%)  | 8(4%) | 2,502(2%) | 3,932(7%) | 1(0%)    | 22(10%) | 5,712(5%) | 11,770(22%) |
| 12             | 1,335                  | 382      | 3.49× | 3(1%)  | 8(4%) | 2,506(2%) | 3,951(7%) | 1(0%)    | 22(10%) | 5,717(5%) | 11,789(22%) |
| 16             | 1,771                  | 502      | 3.53× | 4(1%)  | 8(4%) | 2,477(2%) | 3,936(7%) | 1(0%)    | 22(10%) | 5,719(5%) | 11,782(22%) |
| 24             | 2,643                  | 742      | 3.56× | 4(1%)  | 8(4%) | 2,481(2%) | 3,959(7%) | 1(0%)    | 22(10%) | 5,723(5%) | 11,801(22%) |
| 32             | 3,547                  | 982      | 3.61× | 6(2%)  | 8(4%) | 2,453(2%) | 3,924(7%) | 1(0%)    | 22(10%) | 5,726(5%) | 11,795(22%) |
| 64             | 7,067                  | 1,942    | 3.64× | 6(2%)  | 8(4%) | 2,460(2%) | 3,922(7%) | 2(1%)    | 22(10%) | 5,731(5%) | 11,789(22%) |
| 128            | 14,107                 | 3,990    | 3.54× | 7(3%)  | 8(4%) | 2,467(2%) | 3,921(7%) | 3(1%)    | 22(10%) | 5,709(5%) | 11,751(22%) |
| 256            | 28,187                 | 7,958    | 3.54× | 9(3%)  | 8(4%) | 2,474(2%) | 3,920(7%) | 5(2%)    | 22(10%) | 5,724(5%) | 11,770(22%) |
| 512            | 56,347                 | 15,894   | 3.55× | 14(5%) | 8(4%) | 2,481(2%) | 3,925(7%) | 5(2%)    | 22(10%) | 5,731(5%) | 11,778(22%) |
| 1,024          | 112,667                | 31,766   | 3.55× | 26(9%) | 8(4%) | 2,488(2%) | 3,924(7%) | 6(2%)    | 22(10%) | 5,738(5%) | 11,785(22%) |

The ratio is the speedup of FxP < 8,4 > and FP.

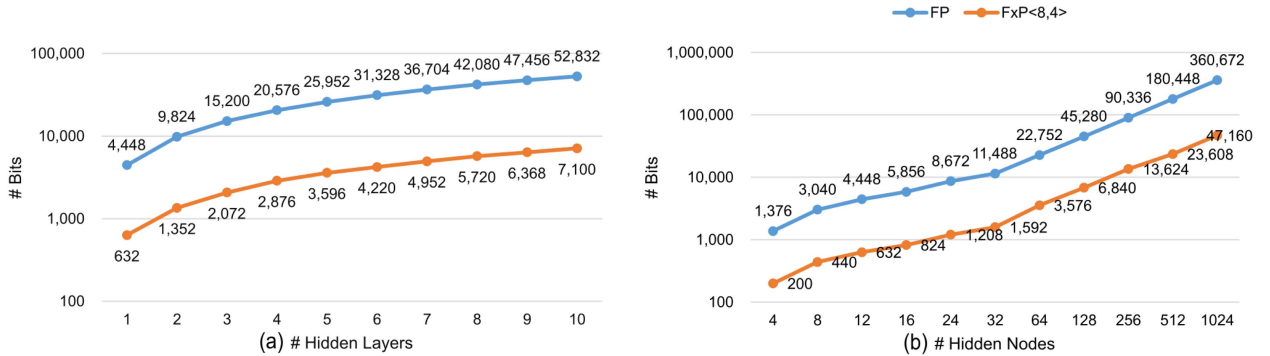


Fig. 10. Memory requirements based on the FP and FxP <8,4> data formats when increasing the number of the hidden layers or the nodes of the MLP models.

format conversion consume a fixed number of clock cycles, the speedup due to the use of the FxP data format converges to about  $3.6\times$  as the number of hidden layers/nodes increases.

In terms of memory requirements, increasing the number of hidden layers or nodes increases the memory requirements for both data representations. As shown in Fig. 10, using the FxP <8,4> data format consumes more than seven times less memory than using the FP data format. Such a reduction in memory bits directly benefits resource consumption. The consumption of BRAM is one of the indicators of the model's memory need because BRAM is dedicated on-chip memory used to store large amounts of memory. BRAM increases due to the increment of FP layers because the weight parameters of a layer require a contiguous block of memory. However, in the case of FxP hidden layers, there is no noticeable increase in BRAM

utilization since FF and LUTs, another type of FPGA resources that can act as distributed memory, are efficient resources used as short memories. In the FxP format, since the bits in the integer part are redundant, only four fractional bits are retained to represent the value on the weight matrix. This short memory need can be efficiently stored with allocated LUT without additional BRAM. In the case of increasing the number of hidden nodes, the large size of the weight matrix increases the consumption of BRAM since LUTs are not sufficient to accommodate the large matrix.

The consumption of FFs and LUTs increases with the number of hidden layers. Because each hidden layer in MLP has an inference matrix and needs a memory area with a particular entry, more FF and LUT are required for the memory address, data fetch, and loop control operations toward the increased



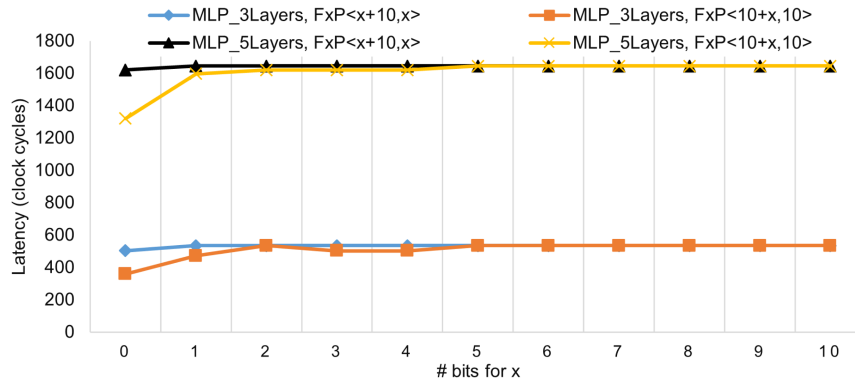


Fig. 11. Latency comparison of two MLP models with varying bit-widths on the integer and fractional part.

number of hidden layers. Meanwhile, the MLP in FxP data format requires fewer bits than their FP data format model, resulting in a lower FF and LUT increment.

Despite requiring fewer bits than FP, the FxP format consumes more FF, LUT, and DSP due to data transfer and format conversion. The model in FxP has more steps in data format conversion than the model in FP. Except for MLP computation, converting data into FxP format also takes resources. Moreover, FP data representation exploits the internal architecture of the DSP components, in which internal stages are designed to process FP numbers in pipeline optimization.

These results highlight the need to carefully consider model topology when designing MLP models on FPGAs to optimize performance and resources. They also reflect the importance of selecting the bit-width and data representation and its impact on latency and resource consumption.

2) *Evaluation of Data Representation: The simulator & profiler* component of the framework enables the selection of the bit-width to reduce latency and resource consumption at the expense of accuracy. As previously shown, using FxP data format for MLP models on FPGA are more than three times faster than using an FP format while demanding fewer memory resources. However, this data representation has a cost in accuracy (Table III). Properly selecting the bit-width and the FxP representation minimizes the impact on accuracy. Our framework evaluates different FxP data representations regarding resource consumption, latency, and accuracy by exploiting the *simulator & profiler*.

Our experiments evaluate the impact of the number of bits on the integer and fractional parts of the FxP data representation. The proposed MLP models are used to evaluate different FxP representations. Unlike conducting all bits combinations, two experiments are conducted to measure the required bits on integer and fractional parts. In the first experiment, the number of bits of the fractional part is modified while keeping the number of bits in the integer part fixed. In the second experiment, the number of bits of the fractional part remains fixed while the number of bits of the integer part changes. Through both experiments, it is possible to identify the optimal FxP data format, which minimizes the degradation in accuracy while offering all the benefits of the FxP representation.

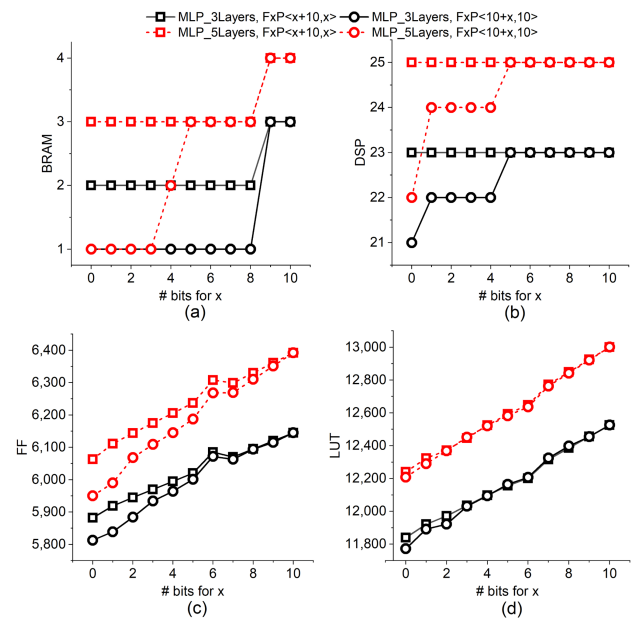


Fig. 12. Resources comparison of MLP\_5Layers (red) and MLP\_3Layers (black) for changing bits. The square symbol represents changing integer bits with fixed fractional bits, while the round symbol represents changing fractional bits with fixed integer bits.

The resource consumption of both experiments is depicted in Fig. 12. Square symbols are the values obtained when varying integer bits with fixed fractional bits of ten, while the round symbols are the values when changing fractional bits with integer bits set to ten. The horizontal axis represents the number of bits  $x$  used for the integer (FxP  $\langle x+10, x \rangle$ ) or the fractional part (FxP  $\langle 10+x, 10 \rangle$ ), meaning that vertical symbols have the same bit-width. Dedicating the same  $x$  value to the integer or the fractional part leads to lower resource consumption, especially in BRAM and DSP consumption. This resource consumption converges after a certain number of bits  $x$ . The model's size defines the convergence speed, occurring earlier for larger models (e.g., MLP\_5Layers). For instance, the amount of BRAM for MLP\_3Layers converges when the bit-width increases to 19 bits (additional 9 bits are used either for the integer or the fractional

TABLE VI  
ACCURACY AND ACCURACY LOSS (ABSOLUTE DIFFERENCE BETWEEN THE FULLY-ACCURATE) COMPARISON OF TWO PROPOSED MLP MODELS WHEN CHANGING BIT-WIDTHS OF INTEGER PART (Fxp <X+20,X>)

| Model<br>Dataset<br>x | MLP_3layers |              |          |              | MLP_5layers |              |          |              |
|-----------------------|-------------|--------------|----------|--------------|-------------|--------------|----------|--------------|
|                       | AFDB        |              | CPSC2018 |              | AFDB        |              | CPSC2018 |              |
|                       | Accuracy    | Loss         | Accuracy | Loss         | Accuracy    | Loss         | Accuracy | Loss         |
| 0                     | 45.02%      | 48.27%       | 56.21%   | 41.80%       | 50.13%      | 44.07%       | 48.73%   | 49.50%       |
| 1                     | 47.49%      | 45.80%       | 27.22%   | 70.79%       | 52.91%      | 41.29%       | 36.70%   | 61.53%       |
| 2                     | 50.21%      | 43.08%       | 56.49%   | 41.52%       | 29.94%      | 64.26%       | 49.00%   | 49.22%       |
| 3                     | 91.05%      | 2.23%        | 97.84%   | 0.17%        | 79.01%      | 15.20%       | 84.15%   | 14.08%       |
| 4                     | 93.29%      | <b>0.00%</b> | 98.00%   | <b>0.00%</b> | 94.20%      | <b>0.00%</b> | 98.23%   | <b>0.00%</b> |
| 5                     | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |
| 6                     | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |
| 7                     | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |
| 8                     | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |
| 9                     | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |
| 10                    | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |
| FP                    | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |

TABLE VII  
ACCURACY AND ACCURACY LOSS (ABSOLUTE DIFFERENCE BETWEEN THE FULLY-ACCURATE) COMPARISON OF TWO PROPOSED MLP MODELS WHEN CHANGING BIT-WIDTHS OF THE FRACTIONAL PART (Fxp <20+X,20>)

| Model<br>Dataset<br>x | MLP_3layers |              |          |              | MLP_5layers |              |          |              |
|-----------------------|-------------|--------------|----------|--------------|-------------|--------------|----------|--------------|
|                       | AFDB        |              | CPSC2018 |              | AFDB        |              | CPSC2018 |              |
|                       | Accuracy    | Loss         | Accuracy | Loss         | Accuracy    | Loss         | Accuracy | Loss         |
| 0                     | 59.25%      | 34.04%       | 83.70%   | 14.30%       | 50.00%      | 44.20%       | 50.00%   | 48.23%       |
| 1                     | 74.26%      | 19.03%       | 85.53%   | 12.47%       | 50.00%      | 44.20%       | 50.00%   | 48.23%       |
| 2                     | 82.84%      | 10.45%       | 84.42%   | 13.58%       | 82.86%      | 11.34%       | 68.46%   | 29.77%       |
| 3                     | 83.76%      | 9.53%        | 81.15%   | 16.85%       | 79.14%      | 15.06%       | 86.25%   | 11.97%       |
| 4                     | 91.77%      | <b>1.52%</b> | 95.29%   | <b>2.72%</b> | 91.28%      | <b>2.92%</b> | 96.78%   | <b>1.44%</b> |
| 5                     | 92.92%      | 0.37%        | 97.23%   | 0.78%        | 93.27%      | 0.93%        | 97.73%   | 0.50%        |
| 6                     | 93.20%      | 0.08%        | 97.73%   | 0.28%        | 93.92%      | 0.28%        | 98.17%   | 0.06%        |
| 7                     | 93.28%      | 0.01%        | 97.95%   | 0.06%        | 94.17%      | 0.03%        | 98.12%   | 0.11%        |
| 8                     | 93.27%      | 0.02%        | 97.95%   | 0.06%        | 94.20%      | 0.00%        | 98.06%   | 0.17%        |
| 9                     | 93.27%      | 0.02%        | 98.00%   | 0.00%        | 94.19%      | 0.01%        | 98.17%   | 0.06%        |
| 10                    | 93.28%      | 0.01%        | 98.00%   | 0.00%        | 94.18%      | 0.02%        | 98.23%   | 0.00%        |
| FP                    | 93.29%      | 0.00%        | 98.00%   | 0.00%        | 94.20%      | 0.00%        | 98.23%   | 0.00%        |

part), while 15 bits are enough to converge in consumption for MLP\_5Layers.

On the other hand, increasing the number of bits in the fractional or integer part increases the FF and LUT consumption due to the additional operations during the inference calculation, which includes data fetch, calculation, and register. As expected, MLP\_5Layers consumes more resources than MLP\_3Layers with the same data format and bit-width.

Although a wider bit-width consumes more resources, the latency of an Fxp data format is stable with about 3–4 clock cycles for one iteration in matrix inference. Except for the type without fractional bits, which only consumes 2–3 clock cycles and improves the speed, as shown in Fig. 11. It comes from eliminating the significant fractional part.

The most important metric is the loss of accuracy due to the bit-width variation in the Fxp data representation. This metric is used to select the optimal Fxp representation. Tables VI and VII summarize the achieved accuracy and loss of accuracy when evaluating the integer or the fractional part, respectively. Table VI shows that four integer bits are sufficient to represent an MLP model, although large models present higher accuracy loss due to the bit-width reduction. While three integer bits are sufficient for the small model MLP\_3Layers with no more than 3% loss in accuracy, a larger model like MLP\_5Layers has a loss of accuracy exceeding 14% for the same bit-width. On the other

hand, our results show that the number of bits of the fractional part significantly impacts the accuracy loss. However, a limited number of fractional bits are also sufficient to represent data that meet real-world requirements. Table VII shows that five bits have no more than 1% loss in accuracy while four fractional bits have less than 3% loss in accuracy. As a result, *simulator & profiler* determined the minimum bit width required to ensure acceptable accuracy while minimizing memory and resource consumption.

3) *Evaluation of Architecture Performance*: The ultimate goal of our framework is the acceleration of MLP models on FPGAs, which implies using different optimizations to increase the model's performance. These optimizations target the two nested loops (L1 and L2) computed for each model layer as described in Section II. As previously described in Algorithm 1, the outer loop L1 iterates over the output nodes of the current layer, while the inner loop L2 iterates over the input nodes of the current layer. The FPGA can speed up the loop operations, which are time-consuming when performed repeatedly, by exploiting their inherent parallelism. The *Design-space exploration* component of the framework applies parallel optimizations to accelerate the loop operations.

The four MLP models described in Table III are involved in the performance evaluation. The data format is fixed to Fxp <8,4>, which is determined in the previous section. The comparison of the values summarized in Tables VIII and IX show that the

TABLE VIII  
RESOURCE UTILIZATION OF GENERATED OPTIMIZED MLP ARCHITECTURES ON FPGA USING A CLOCK FREQUENCY OF 100 MHz,  
Fxp <8,4> DATA REPRESENTATION

| Optimized Architectures | MLP_3Layers  |                |                  |                    | MLP_5Layers  |                |                    |                     |
|-------------------------|--------------|----------------|------------------|--------------------|--------------|----------------|--------------------|---------------------|
|                         | BRAM         | DSP            | FF               | LUT                | BRAM         | DSP            | FF                 | LUT                 |
| BL                      | 1(0%)        | 22(10%)        | 5,859(6%)        | 11,794(22%)        | 2(1%)        | 24(11%)        | 6,004(6%)          | 12,193(23%)         |
| $L_{2P}$                | 2(1%)        | 22(10%)        | 5,863(6%)        | 11,830(22%)        | 5(2%)        | 24(11%)        | 6,021(6%)          | 12,267(23%)         |
| $L_{2U}$                | 0(0%)        | 27(12%)        | 6,267(6%)        | 12,374(23%)        | 0(0%)        | 53(24%)        | 6,945(7%)          | 12,687(24%)         |
| $L_{1P} L_{2U}$         | 0(0%)        | 27(12%)        | 6,522(6%)        | 12,518(24%)        | 0(0%)        | 53(24%)        | 7,436(7%)          | 12,919(24%)         |
| $L_{1U} L_{2U}$         | <b>0(0%)</b> | <b>21(10%)</b> | <b>6,271(6%)</b> | <b>15,136(28%)</b> | <b>0(0%)</b> | <b>22(10%)</b> | <b>7,473(7%)</b>   | <b>24,714(46%)</b>  |
| Optimized Architectures | MLP_Saric    |                |                  |                    | MLP_Borghi   |                |                    |                     |
|                         | BRAM         | DSP            | FF               | LUT                | BRAM         | DSP            | FF                 | LUT                 |
| BL                      | 2(1%)        | 22(10%)        | 5,737(5%)        | 11,807(22%)        | 4(1%)        | 24(11%)        | 5,891(6%)          | 12,223(23%)         |
| $L_{2P}$                | 3(1%)        | 22(10%)        | 5,739(5%)        | 11,839(22%)        | 7(3%)        | 24(11%)        | 5,901(6%)          | 12,281(23%)         |
| $L_{2U}$                | 0(0%)        | 27(12%)        | 6,924(7%)        | 14,002(26%)        | 0(0%)        | 147(67%)       | 10,157(10%)        | 16,130(30%)         |
| $L_{1P} L_{2U}$         | 0(0%)        | 27(12%)        | 7,361(7%)        | 14,219(27%)        | 0(0%)        | 147(67%)       | 11,200(11%)        | 16,415(31%)         |
| $L_{1U} L_{2U}$         | <b>0(0%)</b> | <b>21(10%)</b> | <b>6,698(6%)</b> | <b>18,556(35%)</b> | <b>0(0%)</b> | <b>21(10%)</b> | <b>19,454(18%)</b> | <b>93,682(176%)</b> |

The values in bold represent the optimization with the highest performance, while the value in red exceeds the available resources.

TABLE IX  
SPEEDUP FOR GENERATED ARCHITECTURES ON A CPU AND AN FPGA USING Fxp <8,4> DATA REPRESENTATION

| Optimized Architectures | Platform       | Frequency | Speedup       |               |               |                |
|-------------------------|----------------|-----------|---------------|---------------|---------------|----------------|
|                         |                |           | MLP_3Layers   | MLP_5Layers   | MLP_Saric     | MLP_Borghi     |
| BL                      | Zynq (FPGA)    | 100 MHz   | 1.00×         | 1.00×         | 1.00×         | 1.00×          |
| $L_{2P}$                |                |           | 1.39×         | 1.36×         | 1.41×         | 1.46×          |
| $L_{2U}$                |                |           | 3.90×         | 4.08×         | 4.14×         | 5.33×          |
| $L_{1P} L_{2U}$         |                |           | 10.06×        | 15.29×        | 13.88×        | 63.94×         |
| $L_{1U} L_{2U}$         |                |           | <b>25.15×</b> | <b>62.35×</b> | <b>58.42×</b> | <b>517.04×</b> |
| C/C++ Kernels           | Zynq (ARM CPU) | 666.7 MHz | 0.25×         | 0.30×         | 0.32×         | 0.32×          |
|                         | Xeon CPU       | 2.1 GHz   | 2.27×         | 4.13×         | 4.38×         | 6.11×          |

loop's parallelization usually comes at the cost of resources, especially for full unroll loops requiring multiple PEs to achieve simultaneous computation.

Compared to the BL architecture,  $L_{2P}$  adds little registers in  $L_2$  loop operations to break down the computation into multiple stages (read, compute, and write) and execute them concurrently. Each stage could utilize the intermediate result of the last stage. This data flow reduces the process interval to one or two clock cycles and increases data throughput. However, besides the limited increment for FF and LUT, additional BRAM is required for nodes in each hidden layer for frequent data fetches, though they are 1-dimensional structures with low memory requirements.

In addition to the pipeline optimization with sequential execution, the  $L_2$  loop can be fully unrolled ( $L_{2U}$ ) to execute each loop iteration concurrently. Unlike the one-by-one data process flow on reused PE as in BL and  $L_{2P}$ , multiple identical PEs execute data fetch and computation operations simultaneously. This optimization, however, increases the consumption of DSP, FF, and LUT resources, except for BRAM resources, whose dual-port data interface is no longer suitable for concurrent data fetch.

The outer loop  $L_1$  could use the pipeline to the grouped operations in  $L_2$  for further acceleration, as the operations on  $L_{2U}$  are unrolled without loop control. When reusing the  $L_2$  arithmetic unit, the data flow of the  $L_1$  loop in this  $L_{1P} L_{2U}$  architecture has a data wait interval of one clock cycle. The increment of FF and LUT are used as registers for the intermediate result.

The  $L_1$  loop updates the value in neural nodes within a layer. Because of the data independence between nodes in the same layer, the  $L_1$  loop can also be unrolled for maximum throughput. When both loops are fully unrolled ( $L_{1U} L_{2U}$ ), all loop iterations

are executed in parallel, drastically increasing the consumption of FF and LUT resources. As a result, the MLP\_Borghi model requires more LUT resources than the available in the target FPGA when using this optimization (Table VIII).

Fig. 13 shows the time performance of acceleration. Despite the fact that the MLP models are evaluated on an FPGA with a working frequency of 100 MHz, all architectures on FPGA are faster than a low-end ARM CPU at 667 MHz, and the  $L_{1P} L_{2U}$  and  $L_{1U} L_{2U}$  architectures are faster than the HPC implementation.

Increasing the number of layers or nodes in MLP hardware models results in additional latency consumption due to increased computation operations. However, the fully parallel  $L_{1U} L_{2U}$  architecture reduces latency diversity by performing all operations simultaneously. MLP\_5Layers(b) and MLP\_Saric(c) exhibit greater latency than MLP\_3Layer(a) for increased computation operations, except for the  $L_{1U} L_{2U}$  architecture. MLP\_Borghi(d), a five-layer MLP with more hidden nodes than MLP\_5Layers(b), experiences the longest latency among the MLP models.

HPC CPUs optimize for parallel repeated computations, benefiting larger MLP models. Therefore, the CPU run-time is similar to  $L_{2U}$  architecture for MLP\_5Layers(b) and MLP\_Saric(c). On the other hand, MLP\_Borghi(d) with HPC CPU runs faster than the corresponding hardware  $L_{2U}$  architecture. As unroll optimization performs the same computations simultaneously, the pipeline only reduces the computation interval between two operations. Hence, fully accelerated hardware architecture  $L_{1U} L_{2U}$  exhibits similar time performance (200 ns–460 ns in Fig. 13).

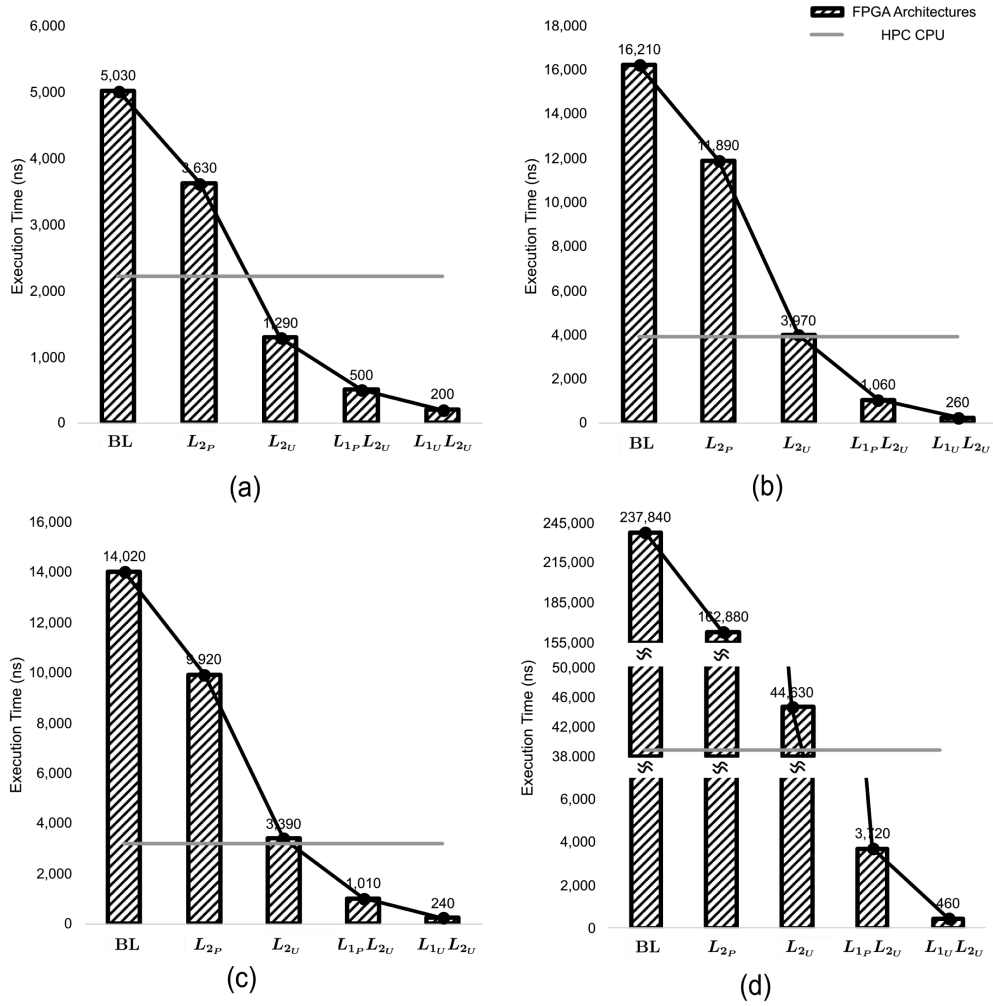


Fig. 13. Time comparison of MLP\_3Layers(a), MLP\_5Layers(b), MLP\_Saric(c), and MLP\_Borghi(d) with HPC CPU.

TABLE X  
POWER AND ENERGY CONSUMPTION FOR GENERATED ARCHITECTURES ON AN ARM CPU AND AN FPGA USING A CLOCK FREQUENCY OF 100 MHz AND FXP <8,4> DATA REPRESENTATION

| Optimized Architectures | Platform       | Frequency | Power consumption  |             |           |              |
|-------------------------|----------------|-----------|--------------------|-------------|-----------|--------------|
|                         |                |           | MLP_3Layers        | MLP_5Layers | MLP_Saric | MLP_Borghi   |
| BL                      | Zynq (FPGA)    | 100 MHz   | 0.012 W            | 0.015 W     | 0.013 W   | 0.017 W      |
| $L_{1u}L_{2u}$          |                |           | 0.01 W             | 0.012 W     | 0.011 W   | -            |
| C/C++ Kernels           | Zynq (ARM CPU) | 666.7 MHz | 1.526 W            | 1.526 W     | 1.526 W   | 1.526 W      |
| Optimized Architectures | Platform       | Frequency | Energy consumption |             |           |              |
|                         |                |           | MLP_3Layers        | MLP_5Layers | MLP_Saric | MLP_Borghi   |
| BL                      | Zynq (FPGA)    | 100 MHz   | 60.36 nJ           | 243.15 nJ   | 182.26 nJ | 404.26 nJ    |
| $L_{1u}L_{2u}$          |                |           | 2.00 nJ            | 3.12 nJ     | 2.64 nJ   | -            |
| C/C++ Kernels           | Zynq (ARM CPU) | 666.7 MHz | 31,299 nJ          | 83,455 nJ   | 67,305 nJ | 1,152,072 nJ |

The values have been obtained for the baseline and the solution with the highest acceleration ( $L_{1p}L_{2u}$ ) for the MLP models under test.

The achieved speedup of each optimized architecture is summarized in Table IX. The impact of the optimizations is similar for all the models, although it is slightly higher for larger models like MLP\_Borghi. That model achieves the highest speedup, but its resource consumption overcomes the available LUT resources of the target FPGA. Nonetheless, two orders of magnitude speedups are achieved when the ML models are accelerated on the FPGA, largely outperforming the CPUs' performances.

Vivado analyzes dynamic power consumption based on the implemented design. The power parameter of the low-end CPU is 1.526 W. FPGA core power depends on resource consumption. As shown in Table X, FPGA has much lower power consumption than CPU. And complex topology implementation has more power consumption than simple topology on FPGA. However, parallel architectures have lower power than BL architectures, and the power reduction comes from resource differences. Moreover, MLP accelerated on FPGA is faster than a wearable CPU.



TABLE XI  
RELATED WORK OF MLP IMPLEMENTED ON FPGAS AND THEIR TIME PERFORMANCE

| Model Type         | Year        | Tools              | Hardware Platform               | Data Format    | Fields          | Parallel | Time          |
|--------------------|-------------|--------------------|---------------------------------|----------------|-----------------|----------|---------------|
| MLP [66]           | 1998        | Viewsynthesis      | -                               | -              | -               | -        | -             |
| MLP-BP [67]        | 2007        | Xilinx ISEe        | XC2v2000 Xilinx Virtex II Pro   | FxP            | -               | ✓        | -             |
| MLP [68]           | 2007        | Xilinx ISEe        | XC2v3000 Xilinx Virtex II       | -              | Gas             | ✓        | 26 ms         |
| MLP-BP [44]        | 2011        | Vivado             | Xilinx Virtex-5 SX50T           | FxP            | -               | ✓        | -             |
| MLP [69]           | 2016        | Vivado             | Zynq-7000-XC7Z010               | 24 bits FxP    | Gas             | ✓        | 0.54 us       |
| MLP [43]           | 2018        | Vivado             | Xilinx Zynq XC7Z020             | FP             | -               | ✓        | 3.5 ms        |
| MLP [65]           | 2020        | ModelSim           | Altera D2-115 Cyclone IV        | 18 bits FxP    | Epilepsy        | -        | -             |
| MLP [49]           | 2020        | Matlab             | Nexys4 Artix7                   | -              | AF              | -        | -             |
| MLP-BP [70]        | 2020        | Vivado             | Xilinx PYNQ-Z1                  | 32 bits FxP    | Images          | -        | 1 ms          |
| MLP [71]           | 2021        | Vivado             | Xilinx Zynq XC7Z020             | FP             | Images          | ✓        | 1.55 us       |
| MLP [72]           | 2023        | Vivado             | Xilinx Artix7                   | 16 bits FxP    | Wearable device | -        | 280 ns        |
| <b>MLP_3Layers</b> | <b>2023</b> | <b>Vivado, HLS</b> | <b>Xilinx PYNQ-Z1, Xeon CPU</b> | <b>FP, FxP</b> | <b>AF</b>       | <b>✓</b> | <b>200 ns</b> |
| <b>MLP_5Layers</b> | <b>2023</b> | <b>Vivado, HLS</b> | <b>Xilinx PYNQ-Z1, Xeon CPU</b> | <b>FP, FxP</b> | <b>AF</b>       | <b>✓</b> | <b>260 ns</b> |

As a result, The computation power is extremely low compared with the CPU, which is essential for wearable devices.

## V. DISCUSSION

MLP is computationally intensive, limiting real-time performance and throughput, which benefits from the unique characteristics of FPGA. As shown in Table XI, MLP models on FPGAs are useful for real-time applications such as gas detection [44], [69], epileptic seizure disease monitoring [65], and anomaly detection [70]. Unfortunately, designing MLP on FPGA is a complex and time-consuming challenge. In addition to the model topology, the data format and the appropriate acceleration determine the FPGA-accelerated MLP performance, accuracy, and consumption of resources. The proposed framework automatically generates MLP accelerators on FPGAs, which have been used to accelerate MLP for AF detection applications.

Our experiments have exploited our framework to evaluate the fundamental mechanisms of MLP models when ported to an FPGA. We have shown how the model topology significantly impacts resource consumption and that combining data representation with concurrency can create an optimized MLP accelerator. Because of their increasing memory requirements as the number of hidden layers and nodes rises, MLP models with complex topologies typically use more resources and suffer from longer latencies (in Section IV-B1). The FxP data format uses fewer memory bits and has a shorter operation interval than the FP data format. In addition, the FxP data format has less gradient increase in resources than the FP data format. Nevertheless, because of the additional data scaling, format conversion, and default support for FP on DSP on modern FPGA, the input data transmission and format conversion of FxP would use significantly more resources than FP. Although the FxP data format significantly impacts the MLP model's accuracy, our *Simulator & Profiler* can assist in identifying an acceptable FxP data format to avoid accuracy loss (in Section IV-B2).

The acceleration of MLP models on FPGAs is possible thanks to exploit optimizations such as pipeline and unrolling of loops. However, reducing the execution time of the MLP models comes with a cost in resource consumption. The pipeline adds registers to the processing module to divide the stages so data can be

processed concurrently. As a result, the pipeline shortens the data processing interval and is especially useful when the process has a high latency. The unroll optimization replaces the loop body with numerous duplicates of the loop body to process data in parallel and has a large throughput with massive resource usage. A careful selection of the available optimizations enables MLP accelerators to satisfy the real-time application's needs while offering acceptable accuracy and resource consumption (Table VIII in Section IV-B3).

The acceleration of model topology affects time improvement, with complex topologies taking longer latency than simpler ones despite higher speedup in optimized architectures. However, fully accelerated  $L_{1U}L_{2U}$  architectures enable neural network forward computation simultaneously and have similar latency regardless of the difference in BL architectures without acceleration (Fig. 13 in Section IV-B3). HPC CPUs are equipped with vectorized instructions to enable parallel computation of neural networks. Additional acceleration is required for MLP on FPGA with complex topologies (repeated operations) to achieve high levels of parallelism comparable to HPC CPUs.

The power report measured by Vivado indicates that specific parallel architectures exhibit low power consumption (Table X in Section IV-B3), a crucial factor for wearable applications. Omitting the use of BRAM can lead to reduced computation energy and power consumption. While this study did not specifically focus on low-power optimization, the efficiency of FPGA still highlights its potential benefits for battery-driven applications [73].

A deployment framework is crucial for accelerating real-time and near real-time applications using MLP on FPGA, as demonstrated in various studies [44], [49], [65], [69], [70], [72] on Table XI. Due to the diverse requirements for MLP acceleration, a comprehensive design framework that considers both MLP and FPGA characteristics is crucial. This work goes beyond existing research by presenting a comprehensive design framework considering both MLP and FPGA properties. In contrast to previous work that utilizes dynamic configuration on pre-deployed MLP on FPGA, the proposed framework designs FPGA based on specific MLP topology and requirements. Moreover, the toolchains for the proposed framework can automatically generate designs to save time and effort [74].

TABLE XII  
COMPARISON OF FRAMEWORKS: DEPLOYMENT AND IMPLEMENTATION

| Works          | Automatic Framework | Architecture Optimization | Design-space Exploration | Implementation Platforms | Models Zoo |
|----------------|---------------------|---------------------------|--------------------------|--------------------------|------------|
| TensorFlow     | ✓                   | ✗                         | ✗                        | CPU, GPU                 | ✓          |
| PyTorch        | ✓                   | ✗                         | ✗                        | CPU, GPU                 | ✓          |
| CNN [39]       | ✗                   | ✓                         | ✗                        | FPGA                     | ✗          |
| CNN [40]       | ✗                   | ✓                         | ✗                        | FPGA                     | ✗          |
| ANNet [41]     | ✗                   | ✓                         | ✗                        | FPGA                     | ✗          |
| DaDianNao [75] | ✓                   | ✗                         | ✗                        | ASIC                     | ✓          |
| hls4ml [76]    | ✓                   | ✓                         | ✗                        | FPGA                     | ✓          |
| Caffeine [77]  | ✓                   | ✓                         | ✗                        | FPGA                     | ✗          |
| AutoMLP        | ✓                   | ✓                         | ✓                        | CPU, FPGA                | ✗          |

As shown in Table XII, user-friendly software frameworks have played an essential role in the widespread adoption of Artificial Intelligence. In real-world scenarios, particularly in the biomedical field, real-time detection and power consumption are critical metrics. Hardware implementations offer improved computational efficiency, making them well-suited for real-time biomedical applications, especially those involving wearable devices [31], [39], [40], [41]. However, designing efficient hardware architectures on FPGAs or application-specific integrated circuits (ASICs) is a complex task considering design preferences and requiring technical expertise [38]. Therefore, it is imperative to research appropriate architectural design to accommodate specific preferences. In addition, developing automatic design toolchains and tutorials for design selection according to target preferences is essential.

In summary, the proposed framework generates MLP accelerators on FPGA suitable for real-time applications. Compared with the present software implemented work in Table I, the accelerated MLP detectors on FPGA are fast with comparable 96.78% accuracy. Compared with the work in Table I, our proposed MLP models have outstanding real-time performance (0.20 us) in AF detection, even compared to simple statistical methods (1.47 ms). Time performance is also essential for big data, which means it could process data at high throughput.

## VI. CONCLUSION

This article introduces AutoMLP, a framework designed to accelerate MLP models on FPGAs. AutoMLP consists of a design flow and automatic toolchain that selects appropriate data bit-widths and data formats for increased performance. Our experiments focused on implementing MLP models for AF disease detection, enabling evaluation in accuracy, resources, and execution time. Our results demonstrate that data representation significantly impacts accuracy and that the FxP data format MLP is three times faster than the FP data format model while maintaining similar accuracy and resource efficiency. AutoMLP also brings multiple optimizations to adjust the design-space exploration toward optimal solutions. The generated MLP accelerators can classify AF in 0.2 us, about two orders of magnitude faster than a high-performance CPU. As a result, AutoMLP enables the generation of hardware MLP accelerators that meet the real-time requirements of demanding applications such as AF disease detection while remaining low-power [78].

## REFERENCES

- [1] P. Kirchhof et al., "2016 ESC Guidelines for the management of atrial fibrillation developed in collaboration with EACTS," *Kardiologia Polska*, vol. 74, no. 12, pp. 1359–1469, 2016.
- [2] P. A. Wolf, R. D. Abbott, and W. B. Kannel, "Atrial fibrillation as an independent risk factor for stroke: The Framingham Study," *Stroke*, vol. 22, no. 8, pp. 983–988, 1991.
- [3] G. Noelker et al., "Performance of an implantable cardiac monitor to detect atrial fibrillation: Results of the DETECT AF study," *J. Cardiovasc. Electrophysiol.*, vol. 27, no. 12, pp. 1403–1410, 2016.
- [4] W. M. Feinberg et al., "Relationship between prothrombin activation fragment F1. 2 and international normalized ratio in patients with atrial fibrillation," *Stroke*, vol. 28, no. 6, pp. 1101–1106, 1997.
- [5] C. T. January et al., "2014 AHA/ACC/HRS guideline for the management of patients with atrial fibrillation: A report of the American College of Cardiology/American Heart Association Task Force on practice guidelines and the heart rhythm society," *J. Amer. College Cardiol.*, vol. 64, no. 21, pp. e1–e76, 2014.
- [6] K. H. Humphries et al., "New-onset atrial fibrillation: Sex differences in presentation, treatment, and outcome," *Circulation*, vol. 103, no. 19, pp. 2365–2370, 2001.
- [7] J. P. DiMarco and J. T. Philbrick, "Use of ambulatory electrocardiographic (Holter) monitoring," *Ann. Intern. Med.*, vol. 113, no. 1, pp. 53–68, 1990.
- [8] M. Grond et al., "Improved detection of silent atrial fibrillation using 72-hour Holter ECG in patients with ischemic stroke: A prospective multicenter cohort study," *Stroke*, vol. 44, no. 12, pp. 3357–3364, 2013.
- [9] S. Nemat et al., "Monitoring and detecting atrial fibrillation using wearable technology," in *Proc. IEEE 38th Annu. Int. Conf. Eng. Med. Biol. Soc.*, 2016, pp. 3394–3397.
- [10] C. C. Cheung, A. D. Krahn, and J. G. Andrade, "The emerging role of wearable technologies in detection of arrhythmia," *Can. J. Cardiol.*, vol. 34, no. 8, pp. 1083–1087, 2018.
- [11] R. Parmar, M. Janveja, J. Pidanic, and G. Trivedi, "Design of DNN-based low-power VLSI architecture to classify atrial fibrillation for wearable devices," *IEEE Trans. Very Large Scale Integration Syst.*, vol. 31, no. 3, pp. 320–330, Mar. 2023.
- [12] Z. Xiao et al., "Atrial fibrillation monitoring based on noncontact capacitive ECG using an integrated microhumidity fabric electrode-sheet sensing scheme," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 4005911.
- [13] K. Grace, J. Salvatier, A. Dafoe, B. Zhang, and O. Evans, "When will AI exceed human performance? Evidence from AI experts," *J. Artif. Intell. Res.*, vol. 62, pp. 729–754, 2018.
- [14] H. Moon et al., "Machine learning enables completely automatic tuning of a quantum device faster than human experts," *Nature Commun.*, vol. 11, no. 1, 2020, Art. no. 4161.
- [15] G. Petmezaz et al., "Automated atrial fibrillation detection using a hybrid CNN-LSTM network on imbalanced ECG datasets," *Biomed. Signal Process. Control*, vol. 63, 2021, Art. no. 102194.
- [16] C. Ma, S. Wei, T. Chen, J. Zhong, Z. Liu, and C. Liu, "Integration of results from convolutional neural network in a support vector machine for the detection of atrial fibrillation," *IEEE Trans. Instrum. Meas.*, vol. 70, 2021, Art. no. 2504610.
- [17] C. Ma et al., "A multistep paroxysmal atrial fibrillation scanning strategy in long-term ECGs," *IEEE Trans. Instrum. Meas.*, vol. 71, 2022, Art. no. 4004010.
- [18] M. Al-dabag, H. S. ALRikabi, and R. Al-Nima, "Anticipating atrial fibrillation signal using efficient algorithm," *Int. J. Online Eng.*, vol. 17, no. 2, pp. 106–120, 2021.
- [19] S. Savalia and V. Emamian, "Cardiac arrhythmia classification by multi-layer perceptron and convolution neural networks," *Bioengineering*, vol. 5, no. 2, 2018, Art. no. 35.
- [20] S. Liaqat, K. Dashtipour, A. Zahid, K. Assaleh, K. Arshad, and N. Ramzan, "Detection of atrial fibrillation using a machine learning approach," *Information*, vol. 11, no. 12, 2020, Art. no. 549.
- [21] L. Da Xu, W. He, and S. Li, "Internet of things in industries: A survey," *IEEE Trans. Ind. Informat.*, vol. 10, no. 4, pp. 2233–2243, Nov. 2014.
- [22] C.-Y. Chou, E.-J. Chang, H.-T. Li, and A.-Y. Wu, "Low-complexity privacy-preserving compressive analysis using subspace-based dictionary for ECG telemonitoring system," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 4, pp. 801–811, Aug. 2018.
- [23] K. Ovcharov, O. Ruwase, J.-Y. Kim, J. Fowers, K. Strauss, and E. S. Chung, "Accelerating deep convolutional neural networks using specialized hardware," *Microsoft Res. Whitepaper*, vol. 2, no. 11, pp. 1–4, 2015.

- [24] C. Chen, B. da Silva, R. Chen, S. Li, J. Li, and C. Liu, "Evaluation of fast sample entropy algorithms on FPGAs: From performance to energy efficiency," *Entropy*, vol. 24, no. 9, 2022, Art. no. 1177.
- [25] M. Ling et al., "Vina-FPGA: A hardware-accelerated molecular docking tool with fixed-point quantization and low-level parallelism," *IEEE Trans. Very Large Scale Integration Syst.*, vol. 31, no. 4, pp. 484–497, Apr. 2023.
- [26] D. Diakite and N. Gac, "X-Ray tomography reconstruction accelerated on FPGA through high-level synthesis tools," *IEEE Trans. Biomed. Circuits Syst.*, vol. 17, no. 2, pp. 375–389, Apr. 2023.
- [27] K. Kara, D. Alistarh, G. Alonso, O. Mutlu, and C. Zhang, "FPGA-accelerated dense linear machine learning: A precision-convergence trade-off," in *Proc. IEEE 25th Annu. Int. Symp. Field-Programmable Custom Comput. Mach.*, 2017, pp. 160–167.
- [28] J. Duarte et al., "FPGA-accelerated machine learning inference as a service for particle physics computing," *Comput. Softw. Big Sci.*, vol. 3, no. 1, pp. 1–15, 2019.
- [29] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2015, pp. 161–170.
- [30] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "FP-BNN: Binarized neural network on FPGA," *Neurocomputing*, vol. 275, pp. 1072–1086, 2018.
- [31] D. L. T. Wong, Y. Li, D. John, W. K. Ho, and C.-H. Heng, "An energy efficient ECG ventricular ectopic beat classifier using binarized CNN for edge AI devices," *IEEE Trans. Biomed. Circuits Syst.*, vol. 16, no. 2, pp. 222–232, Apr. 2022.
- [32] Y. Li, Z. Wang, B. Ding, and C. Zhang, "AutoML: A perspective where industry meets academy," in *Proc. 27th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 4048–4049.
- [33] H. Jin, Q. Song, and X. Hu, "Auto-Keras: An efficient neural architecture search system," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1946–1956.
- [34] M. Feurer, A. Klein, K. Eggenberger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2962–2970.
- [35] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Comput. Surv.*, vol. 55, no. 6, pp. 1–29, 2022.
- [36] H. Son et al., "A multilayer-learning current-mode neuromorphic system with analog-error compensation," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 5, pp. 986–998, Oct. 2019.
- [37] K. Zhao, H. Jiang, Z. Wang, P. Chen, B. Zhu, and X. Duan, "Long-term bowel sound monitoring and segmentation by wearable devices and convolutional neural networks," *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 5, pp. 985–996, Oct. 2020.
- [38] Y. Wei et al., "A review of algorithm & hardware design for AI-based biomedical applications," *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 2, pp. 145–163, Apr. 2020.
- [39] S.-Y. Lee, Y.-W. Hung, Y.-T. Chang, C.-C. Lin, and G.-S. Shieh, "RISC-V CNN coprocessor for real-time epilepsy detection in wearable application," *IEEE Trans. Biomed. Circuits Syst.*, vol. 15, no. 4, pp. 679–691, Aug. 2021.
- [40] P. Peng et al., "Design of an efficient CNN-Based cough detection system on lightweight FPGA," *IEEE Trans. Biomed. Circuits Syst.*, vol. 17, no. 1, pp. 116–128, Feb. 2023.
- [41] G. Sivapalan, K. K. Nundy, S. Dev, B. Cardiff, and D. John, "ANNet: A lightweight neural network for ECG anomaly detection in IoT edge sensors," *IEEE Trans. Biomed. Circuits Syst.*, vol. 16, no. 1, pp. 24–35, Feb. 2022.
- [42] T. Chen et al., "BenchNN: On the broad potential application scope of hardware neural network accelerators," in *Proc. IEEE Int. Symp. Workload Characterization*, 2012, pp. 36–45.
- [43] Z. Hajduk, "Reconfigurable FPGA implementation of neural networks," *Neurocomputing*, vol. 308, pp. 227–234, 2018.
- [44] A. Gomperts, A. Ukil, and F. Zurluf, "Development and implementation of parameterized FPGA-based general purpose neural networks for on-line applications," *IEEE Trans. Ind. Informat.*, vol. 7, no. 1, pp. 78–89, Feb. 2011.
- [45] L. Zhao, C. Liu, S. Wei, Q. Shen, F. Zhou, and J. Li, "A new entropy-based atrial fibrillation detection method for scanning wearable ECG recordings," *Entropy*, vol. 20, no. 12, 2018, Art. no. 904.
- [46] S. Saadatnejad, M. Oveis, and M. Hashemi, "LSTM-based ECG classification for continuous monitoring on personal wearable devices," *IEEE J. Biomed. Health Inform.*, vol. 24, no. 2, pp. 515–523, Feb. 2020.
- [47] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH arrhythmia database," *IEEE Eng. Med. Biol. Mag.*, vol. 20, no. 3, pp. 45–50, May/Jun. 2001.
- [48] H. Zhang, Z. Dong, J. Gao, P. Lu, and Z. Wang, "Automatic screening method for atrial fibrillation based on lossy compression of the electrocardiogram signal," *Physiol. Meas.*, vol. 41, no. 7, 2020, Art. no. 075005.
- [49] H. Zairi, M. K. Talha, K. Meddah, and S. O. Slimane, "FPGA-based system for artificial neural network arrhythmia classification," *Neural Comput. Appl.*, vol. 32, no. 8, pp. 4105–4120, 2020.
- [50] S. Mishra, H. K. Tripathy, P. K. Mallick, A. K. Bhoi, and P. Barsocchi, "EAGA-MLP—an enhanced and adaptive hybrid classification model for diabetes diagnosis," *Sensors*, vol. 20, no. 14, 2020, Art. no. 4036.
- [51] P. H. Borghi, R. C. Borges, and J. P. Teixeira, "Atrial fibrillation classification based on MLP networks by extracting Jitter and Shimmer parameters," *Procedia Comput. Sci.*, vol. 181, pp. 931–939, 2021.
- [52] G. G. Geweid and J. D. Chen, "Automatic classification of atrial fibrillation from short single-lead ECG recordings using a hybrid approach of dual support vector machine," *Expert Syst. Appl.*, vol. 198, 2022, Art. no. 116848.
- [53] M. S. Manikandan and K. Soman, "A novel method for detecting R-peaks in electrocardiogram (ECG) signal," *Biomed. Signal Process. Control*, vol. 7, no. 2, pp. 118–128, 2012.
- [54] C. Chen et al., "An atrial fibrillation detection system based on machine learning algorithm with mix-domain features and hardware acceleration," in *Proc. IEEE 43rd Annu. Int. Conf. Eng. Med. Biol. Soc.*, 2021, pp. 1423–1426.
- [55] T. Tekeste, H. Saleh, B. Mohammad, A. Khandoker, H. Jelinek, and M. Ismail, "A nanowatt real-time cardiac autonomic neuropathy detector," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 4, pp. 739–750, Aug. 2018.
- [56] J. S. Richman and J. R. Moorman, "Physiological time-series analysis using approximate entropy and sample entropy," *Amer. J. Physiol. Heart Circulatory Physiol.*, vol. 278, no. 6, pp. H2039–H2049, 2000.
- [57] D. E. Lake and J. R. Moorman, "Accurate estimation of entropy in very short physiological time series: The problem of atrial fibrillation detection in implanted ventricular devices," *Amer. J. Physiol. Heart Circ. Physiol.*, vol. 300, no. 1, pp. H319–H325, 2011.
- [58] S. Buś, K. Jedrzejewski, and P. Guzik, "Using minimum redundancy maximum relevance algorithm to select minimal sets of heart rate variability parameters for atrial fibrillation detection," *J. Clin. Med.*, vol. 11, no. 14, 2022, Art. no. 4004.
- [59] V. Krasteva, I. Christov, S. Naydenov, T. Stoyanov, and I. Jekova, "Application of dense neural networks for detection of atrial fibrillation and ranking of augmented ECG feature set," *Sensors*, vol. 21, no. 20, 2021, Art. no. 6848.
- [60] I. Jekova, I. Christov, and V. Krasteva, "Atrioventricular synchronization for detection of atrial fibrillation and flutter in one to twelve ECG leads using a dense neural network classifier," *Sensors*, vol. 22, no. 16, 2022, Art. no. 6071.
- [61] R. Rouhi, M. Clausel, J. Oster, and F. Lauer, "An interpretable hand-crafted feature-based model for atrial fibrillation detection," *Front. Physiol.*, vol. 12, 2021, Art. no. 657304.
- [62] G. Moody, "A new method for detecting atrial fibrillation using RR intervals," in *Proc. Comput. Cardiol.*, 1983, pp. 227–230.
- [63] A. L. Goldberger et al., "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
- [64] F. Liu et al., "An open access database for evaluating the algorithms of electrocardiogram rhythm and morphology abnormality detection," *J. Med. Imag. Health Inform.*, vol. 8, no. 7, pp. 1368–1373, 2018.
- [65] R. Sarić, D. Jokić, N. Beganović, L. G. Pokvić, and A. Badnjević, "FPGA-based real-time epileptic seizure classification using artificial neural network," *Biomed. Signal Process. Control*, vol. 62, 2020, Art. no. 102106.
- [66] Y. Taright and M. Hubin, "FPGA implementation of a multilayer perceptron neural network using VHDL," in *Proc. IEEE 4th Int. Conf. Signal Process.*, 1998, pp. 1311–1314.
- [67] A. W. Savich, M. Moussa, and S. Areibi, "The impact of arithmetic representation on implementing MLP-BP on FPGAs: A study," *IEEE Trans. Neural Netw.*, vol. 18, no. 1, pp. 240–252, Jan. 2007.
- [68] M. Shi, S. Chandrasekaran, A. Bermak, and A. Amira, "FPGA based run time reconfigurable gas discrimination system," in *Proc. IEEE Int. Symp. Integr. Circuits*, 2007, pp. 180–183.
- [69] X. Zhai, A. A. S. Ali, A. Amira, and F. Bensaali, "MLP neural network based gas classification system on Zynq SoC," *IEEE Access*, vol. 4, pp. 8138–8146, 2016.



- [70] M. Tsukada, M. Kondo, and H. Matsutani, "A neural network-based on-device learning anomaly detector for edge devices," *IEEE Trans. Comput.*, vol. 69, no. 7, pp. 1027–1044, Jul. 2020.
- [71] I. Westby, X. Yang, T. Liu, and H. Xu, "FPGA acceleration on a multi-layer perceptron neural network for digit recognition," *J. Supercomput.*, vol. 77, no. 12, pp. 14356–14373, 2021.
- [72] N. B. Gaikwad et al., "Hardware design and implementation of Multi-Agent MLP regression for the estimation of gunshot direction on IoBT edge gateway," *IEEE Sens. J.*, vol. 23, no. 13, pp. 14549–14557, Jul. 2023.
- [73] A. Jafari, N. Buswell, M. Ghovanloo, and T. Mohsenin, "A low-power wearable stand-alone tongue drive system for people with severe disabilities," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 1, pp. 58–67, Feb. 2018.
- [74] C. Chen, "AutoMLP project," 2022. [Online]. Available: <https://github.com/ChaosBest/AutoMLP>
- [75] T. Luo et al., "DaDianNao: A neural network supercomputer," *IEEE Trans. Comput.*, vol. 66, no. 1, pp. 73–88, Jan. 2017.
- [76] J. Duarte et al., "Fast inference of deep neural networks in FPGAs for particle physics," *J. Instrum.*, vol. 13, no. 07, 2018, Art. no. P07027.
- [77] C. Zhang, G. Sun, Z. Fang, P. Zhou, P. Pan, and J. Cong, "Caffeine: Toward uniform representation and acceleration for deep convolutional neural networks," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 38, no. 11, pp. 2072–2085, Nov. 2019.
- [78] C. Chen, B. D. Silva, J. Li, and C. Liu, "Acceleration of fast sample entropy towards biomedical applications on FPGAs," in *Proc. IEEE Int. Conf. Field-Programmable Technol.*, 2022, pp. 1–4.



**Caiyun Ma** (Graduate Student Member, IEEE) received the M.S. degree in biomedical engineering from Shandong University, Jinan, China, in 2020. She is currently working toward the Ph.D. degree in instrument science and technology with Southeast University, Nanjing, China, where she mainly working on physiological signals processing and the atrial fibrillation signals classification using machine learning



**Jianqing Li** received the B.S. and M.S. degrees in automatic technology from the School of Instrument Science and Engineering, Southeast University, Nanjing, China, in 1986 and 1992 respectively, and the Ph.D. degree in measurement technology and instruments from Southeast University. He is currently a Professor and Vice Present with the School of Basic Medical Sciences, Nanjing Medical University, Nanjing. He is also a Professor with the School of Instrument Science and Engineering, Southeast University. His research interests include mHealth and wireless network.



**Chao Chen** After two years of working as an Electrical Engineer with Nari Group, he returned to academia. He started the Master's studies in 2018 and then, in 2019, began to pursue his Ph.D. with the School of Instrument Science and Engineering, Southeast University. In 2021, he joined the Vrije Universiteit Brussel, Brussels, Belgium, as a Visiting Student. His research interests include domain-specific computing architectures, particularly accelerating biomedical signal processing by applying information entropy and machine learning on FPGAs.

He also has a strong interest in developing wearable algorithms and hardware.



**Chengyu Liu** (Senior Member, IEEE) received the Ph.D. degrees in biomedical engineering from Shandong University, Jinan, China, in 2010. He has completed the Postdoctoral trainings with Shandong University, during 2010–2013, Newcastle University, Newcastle upon Tyne, U.K. during 2013–2014, and Emory University, Atlanta, GA, USA, during 2015–2017. He is currently the Director and Professor of Southeast-Lenovo Wearable Heart-Sleep-Emotion Intelligent Monitoring Lab, School of Instrument Science and Engineering, Southeast University, Nanjing, China. He has authored or coauthored more than 180 papers, including more than 80 SCI journal papers, eight Chapters in Books, more than ten Invention Patents. His main research interests include intelligent monitoring and analysis of vital signs signals (ECG, blood pressure, pulse, respiration, heart sounds), medical Big Data processing and machine learning, research and development of wearable active health monitoring device. He is currently a Federation Journal Committee Member of International Federation for Medical and Biological Engineering.



**Bruno da Silva** (Member, IEEE) received the Ph.D. degree from the Vrije Universiteit Brussel, Brussels, Belgium and Universiteit Gent, Ghent, Belgium, in 2019. Since 2022, he has been an Assistant Professor with the Engineering Sciences Faculty of the Vrije Universiteit Brussel. As a member of the Electronics and Informatics Department (ETRO), he is responsible for research in reconfigurable technologies, low-power biomedical devices, and embedded AI



**Chenxi Yang** (Member, IEEE) received the M.Eng. degree and Ph.D. degree in electrical engineering from the Stevens Institute of Technology, Hoboken, NJ, USA, in 2015 and 2020, respectively. He is currently an Associate Professor with Southeast University, Nanjing, China. His research interests include biophysical signal processing and mobile healthcare with sensor networks. He is a member of IEEE Signal Processing Society, Circuits and Systems Society, and Engineering in Medicine and Biology Society. He was the recipient of the NSF EMBC Award for Young Professionals on Smart and Connected Health at EMBC 2016, Orlando, FL. His research is funded by National Science Foundation of China

His research is funded by National Science Foundation of China